

# Hệ thống đấu giá trực tuyến

## Online Auction System | Team N7 | UET - VNU.

### Phần 1: Mục tiêu, phạm vi thực hiện.

#### Mục tiêu Học thuật

- Ứng dụng nguyên lý OOP nâng cao và Design Patterns bao gồm Singleton, Factory, Observer, và Command.
- Lập trình Socket đa luồng với mô hình Client-Server phân tán.
- Thiết kế kiến trúc phân lớp rõ ràng bao gồm Presentation, Business Logic, và Data Access.

#### Mục tiêu Kỹ thuật

- Độ trễ đồng bộ cực thấp dưới 100 ms qua kết nối Socket TCP.
- Bảo mật 2FA theo phương thức TOTP bằng cách quét mã QR qua Google Authenticator.
- An toàn dòng tiền ví theo chuẩn ACID và chống race condition bằng Memory Synchronized & Triggers.

#### Phạm vi Thực hiện

- Mô hình Client-Server phân tán — Server giữ SQLite, Client giao tiếp qua WebSocket TCP.

#### Ba vai trò nghiệp vụ

- Bidder: Đặt giá thủ công, Auto-Bid tự động, xem biểu đồ giá real-time.
- Seller: Đăng tải hàng hóa, thiết lập cấu hình phiên đấu giá.
- Admin: Phê duyệt phiên, giám sát dòng tiền toàn hệ thống.

#### Công nghệ sử dụng

- Java 25, JavaFX, WebSocket, SQLite, BCrypt, TOTP, Maven, GitHub Actions.

### Phần 2: Kiến trúc tổng thể hệ thống.

#### • Client (JavaFX MVC)

- Giao diện xây dựng bằng JavaFX FXML + CSS.
- Tách biệt hoàn toàn tầng hiển thị và tầng kết nối thông qua AuctionEventBus — giúp cập nhật UI an toàn, không đồng bộ trên luồng JavaFX.
- Các Controller nhận event từ Bus và cập nhật Scene Graph; không gọi Socket trực tiếp.

#### • Networking (WebSocket)

- WebSocket TCP thay thế Socket truyền thống. Gói tin định dạng JSON được mã hóa AES-256-GCM.
- Client gửi Command, Server trả Response qua cùng kênh full-duplex.
- Áp dụng Batching + Debouncing giúp giảm tải truyền real-time cho broadcast giá thầu đồng thời.

#### • Server (Multi-Thread)

- MultiThreadedServer đón nhận kết nối và chia luồng ClientHandler xử lý song song.
- CommandDispatcher định tuyến gói tin JSON đến các Handler chuyên biệt.
- AutoBidEngine chạy nền đặt giá tự động và áp dụng Publish-Subscribe pattern để phát broadcast cập nhật giá tới tất cả Client trong phiên.

#### • Database (SQLite + DAO)

- Đóng gói toàn bộ câu lệnh SQL qua lớp DAO (Data Access Object).
- TransactionManager bọc tác vụ luồng chạy nền, quản lý giao dịch ACID đảm bảo nhất quán dữ liệu.
- Sử dụng Pessimistic Locking và Database Triggers để chống race condition khi nhiều luồng ghi đồng thời.

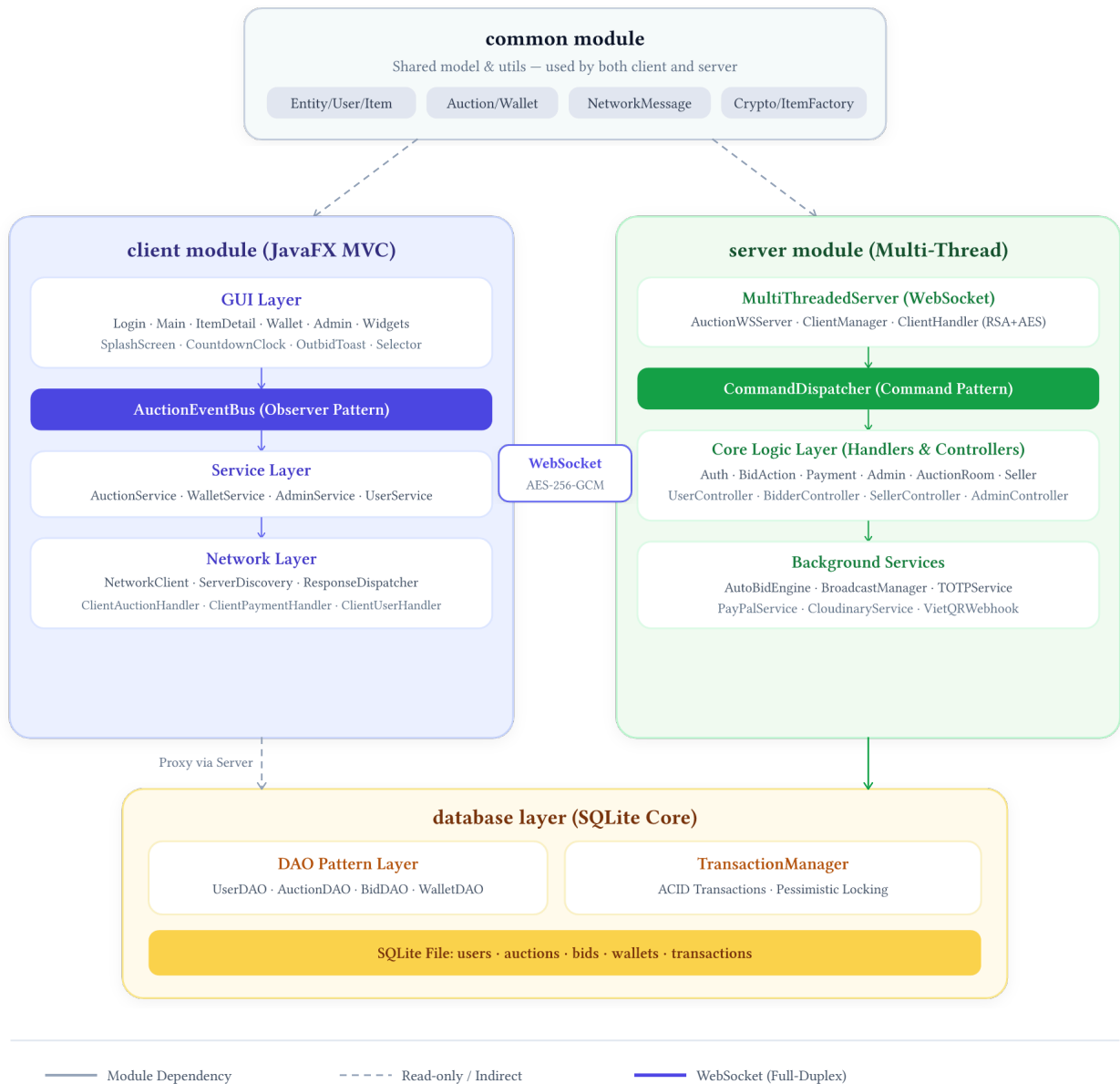


Figure 1: Kiến trúc tổng thể hệ thống N7 Auction System

### Phần 3: Chức năng đạt được theo Barem

| Chức năng / Tiêu chí   | Hướng giải quyết kỹ thuật                                                                                      | Lý do lựa chọn                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Thiết kế OOP & Pattern | Kế thừa lớp cơ sở (Entity, User, Item). Áp dụng Singleton, Factory Method, Observer, và Command Pattern.       | Đảm bảo tính đóng gói, giảm coupling, dễ mở rộng tính năng mà không phá vỡ cấu trúc cũ.   |
| Xử lý lỗi & Ngoại lệ   | Mã hóa lỗi JSON tập trung tại Server. Client bắt lỗi và hiển thị dạng Toast thông báo. Tự động kết nối lại.    | Validate phía Server ngăn chặn can thiệp từ Client, tăng trải nghiệm người dùng.          |
| Xác thực người dùng    | Mã hóa mật khẩu bằng BCrypt. Quản lý phiên bằng Stateful Session duy trì trực tiếp trên WebSocket.             | BCrypt chống tấn công brute-force; Session quản lý chặt chẽ trên kết nối TCP.             |
| Quản lý & Đăng tải     | Hệ thống CRUD phân quyền, cơ chế Kick phiên từ Admin. Seller upload ảnh qua Cloudinary (có nén ảnh từ Client). | Thắt chặt bảo mật, tối ưu băng thông truyền tải và kiểm soát chất lượng sản phẩm đấu giá. |

|                      |                                                                                                                          |                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Cơ chế Đặt giá (Bid) | Kiểm tra điều kiện server-side. Auto-Bid Engine chạy nền dùng Priority Queue dựa trên mô hình Pub-Sub.                   | Ngăn chặn giá giả mạo, tách biệt logic tránh hiện tượng nghẽn luồng (deadlock).                |
| Đồng bộ Real-time    | Sử dụng WebSocket TCP phát broadcast. Áp dụng Debouncing gộp sự kiện (50ms) và AuctionEventBus cho UI.                   | Giảm tải băng thông, tránh tình trạng bão tin nhắn ảnh hưởng đến hiệu năng Client.             |
| Concurrency & ACID   | Chống Race Condition bằng Pessimistic Lock và Database Triggers. TransactionManager bọc tác vụ ghi đa luồng bất đồng bộ. | Giải quyết triệt để lỗi Lost Update trong SQLite đa luồng mà không gây nghẽn cổ chai.          |
| Ví điện tử & Ví 2FA  | Cơ chế khóa số dư tạm thời (Locked balance). Tích hợp PayPal Sandbox API. Xác thực 2FA qua TOTP (Google Authenticator).  | Đảm bảo toàn vẹn dòng tiền chuẩn ACID, nâng cấp bảo mật tài khoản chuẩn RFC 6238.              |
| Tính năng Nâng cao   | Cơ chế Anti-Snipping (giá hạn 120s). Vẽ Line Chart real-time qua JavaFX. Tìm kiếm thông minh Fuzzy Search tiếng Việt.    | Tăng tính công bằng khi đấu giá, trực quan hóa dữ liệu và tối ưu trải nghiệm tìm kiếm.         |
| Chất lượng Mã nguồn  | Cấu trúc đa mô-đun với Maven. Tuân thủ Google Java Style. Viết Unit Test đa luồng bằng CountdownLatch.                   | Mã nguồn sạch, chuẩn hóa quy trình làm việc nhóm và phát hiện sớm các lỗi xung đột luồng.      |
| CI/CD tự động        | Thiết lập GitHub Actions tự động biên dịch, chạy kiểm thử và đóng gói Fat JAR khi có Push hoặc Pull Request.             | Đảm bảo nhánh chính ổn định, không bị lỗi và dễ dàng triển khai sản phẩm chỉ với một câu lệnh. |

#### Phần 4: Phân công công việc giữa các thành viên

- **Cao Dương Lễ (Principal Architect — Bảo mật & Xử lý Song song)**
  - Xử lý Race Condition trong PaymentHandler và AutoBidEngine.
  - Vả lỗ hổng bypass phân quyền Admin, ngăn Seller sửa phiên đã duyệt.
  - Triển khai Pessimistic Lock + Database Triggers ACID.
  - Xây dựng cơ chế Kick phiên làm việc ngay lập tức khi Admin Block người dùng.
  - Phát triển tính năng Anti-Snipping, TOTP 2FA, Quên mật khẩu, Rút tiền.
  - Tối ưu hóa WebSocket với Batching/Debouncing + Publish-Subscribe.
  - Viết Unit Test song song bằng CountdownLatch (giá lập 10 luồng / 1ms).
- **Nguyễn Quang Mạnh (Kỹ sư Tài chính & OOP Core WebSocket)**
  - Tích hợp PayPal Sandbox API: Chống Double-Spending bằng cách kiểm tra PayPal Order ID.
  - Chuẩn hóa tiền tệ: Chuyển đổi toàn bộ kiểu dữ liệu từ double sang long trên toàn hệ thống để tránh sai số.
  - Cập nhật tỷ giá ngoại tệ theo thời gian thực (Real-time).
  - Tái cấu trúc OOP: Chuẩn hóa mô hình phân cấp User -> Bidder / Seller, áp dụng Factory Method cho cấu trúc Item.
  - Nâng cấp hệ thống: Chuyển đổi từ Socket truyền thống sang WebSocket kết hợp mã hóa AES-256-GCM.
  - Tối ưu hiệu năng: Triển khai Non-blocking DB query chống hiện tượng NIO Thread Starvation.
- **Ngô Vũ Đình Khoa (UI/UX Lead — Hệ thống Tìm kiếm & Admin)**
  - Phát triển thuật toán Fuzzy search tiếng Việt có dấu và hỗ trợ tìm kiếm khi sai thứ tự từ.
  - Xây dựng bộ lọc danh mục sản phẩm thông minh (Selector).
  - Thiết kế tính năng cho Admin: Hủy phiên vi phạm, đánh giá uy tín người dùng, xem lịch sử hoạt động của Seller.
  - Refactor kiến trúc MVC: Tách biệt WalletService và ClientUserController.

- Sửa lỗi Memory Leak trong JavaFX và tối ưu hóa cơ chế đồng bộ thời gian giữa Client-Server (giảm từ 10 phút xuống còn 5 giây).
- Trực quan hóa dữ liệu: Vẽ Line Chart hiển thị biến động giá thầu và số dư ví theo thời gian thực.
- Tối ưu giao diện: Hỗ trợ Auto-scale tỷ lệ 0.75, tích hợp nén ảnh phía Client, chuyển đổi ngôn ngữ giao diện sang Tiếng Anh.
- **Khúc Ngọc Minh (Phát triển Giao diện Ban đầu)**
  - Thiết kế cấu trúc giao diện Đăng nhập & Đăng ký ban đầu.
  - Xây dựng giao diện xem chi tiết sản phẩm dành cho người bán (ItemDetailController).
  - Tinh chỉnh tỷ lệ scale của các widget trên giao diện người dùng.

### Giải trình chênh lệch đóng góp

- Sự phân chia phản ánh chân thực nỗ lực đóng góp chất xám kỹ thuật của từng thành viên vào lõi dự án. Phần Back-end (Concurrency, DB Transaction, Socket Server, Fintech) đòi hỏi độ phức tạp giải thuật và chịu rủi ro cao hơn so với thiết kế giao diện Client. Cả nhóm đồng thuận 100% với bảng phân công này, đảm bảo tính công bằng và minh bạch.

### Phần 5: Tự đánh giá theo baren và kết luận.

| Tiêu chí                            | Điểm tối đa | Tự đánh giá | Ghi chú                                               |
|-------------------------------------|-------------|-------------|-------------------------------------------------------|
| Thiết kế lớp & cây kế thừa          | 0.5         | 0.5         | Entity → Item/User → các lớp con chuyên biệt          |
| Áp dụng nguyên tắc OOP              | 1.0         | 1.0         | Encapsulation, Inheritance, Polymorphism, Abstraction |
| Design Pattern                      | 1.0         | 1.0         | Singleton, Factory Method, Observer, Command          |
| Quản lý người dùng & sản phẩm       | 1.0         | 1.0         | CRUD, Block/Kick, upload ảnh, duyệt Admin             |
| Chức năng đấu giá (Manual Bid)      | 1.0         | 1.0         | Validate server-side, Pessimistic Locking             |
| Xử lý lỗi & ngoại lệ                | 1.0         | 1.0         | Validate server-side, Toast notification, reconnect   |
| Concurrent Bidding (Race Condition) | 1.0         | 1.0         | Pessimistic Lock + Triggers SQLite                    |
| Realtime Update (WebSocket)         | 0.5         | 0.5         | Debouncing, EventBus, AES-256-GCM                     |
| Kiến trúc Client-Server             | 0.5         | 0.5         | BCrypt + WebSocket Session                            |
| Kiến trúc MVC + DAO                 | 0.5         | 0.5         | JavaFX FXML client, Controller-Model-DAO server       |
| Maven + Convention + Unit Test      | 1.0         | 1.0         | Google Java Style, JUnit, CountdownLatch stress-test  |
| CI/CD GitHub Actions                | 0.5         | 0.5         | mvn test + mvn package tự động trên mỗi PR            |
| Auto-Bidding                        | 0.5         | 0.5         | Priority Queue, Pub-Sub, chống deadlock               |
| Anti-Sniping                        | 0.5         | 0.5         | Gia hạn 120s khi bid trong 60s cuối                   |
| Bid History Visualization           | 0.5         | 0.5         | JavaFX Line Chart real-time                           |
| Các tính năng khác                  | 0.5         | 0.5         | TOPT, Paypal                                          |
| <b>TỔNG CỘNG</b>                    | <b>10</b>   | <b>10</b>   | <b>10đ bắt buộc + 1.5đ nâng cao =&gt; 10đ</b>         |

## Kết luận và hướng phát triển.

### Bài học Kinh nghiệm

- **Xử lý đa luồng và đồng bộ mạng là hai bài toán phức tạp nhất của dự án:** Nhóm học được rằng Pessimistic Lock bộ nhớ kết hợp Database Triggers cực kỳ hiệu quả và an toàn trong môi trường đa luồng.
- **Tối ưu hóa băng thông:** Áp dụng cơ chế Debouncing và Batching chính là chìa khóa vàng để giảm tải cho hệ thống mạng khi có lượng lớn giá thầu được gửi lên đồng thời.
- **Hiện thực hóa lý thuyết:** Chuẩn ACID không chỉ nằm trên sách vở — từng giao dịch tiền tệ nhỏ nhất đều phải được kiểm thử nghiêm ngặt thông qua các kịch bản stress-test bằng CountdownLatch.

### Hướng Mở rộng

- **Chuyên dịch kiến trúc:** Nâng cấp hệ thống sang kiến trúc Microservices, tách biệt hoàn toàn các dịch vụ độc lập bao gồm Auction Service, Wallet Service, và Notification Service.
- **Nâng cao bảo mật:** Sử dụng giao thức WSS (WebSocket Secure kết hợp TLS) để đảm bảo toàn bộ kết nối được mã hóa đầu cuối (end-to-end).
- **Mở rộng hạ tầng lưu trữ:** Triển khai hệ quản trị cơ sở dữ liệu PostgreSQL thay thế cho SQLite nhằm đáp ứng tốt khi quy mô người dùng tăng trưởng mạnh.
- **Tự động hóa quy trình:** Tích hợp hệ thống CI/CD hoàn chỉnh với GitHub Actions nhằm tự động hóa các khâu từ build, test, đóng gói Docker image cho đến deploy.